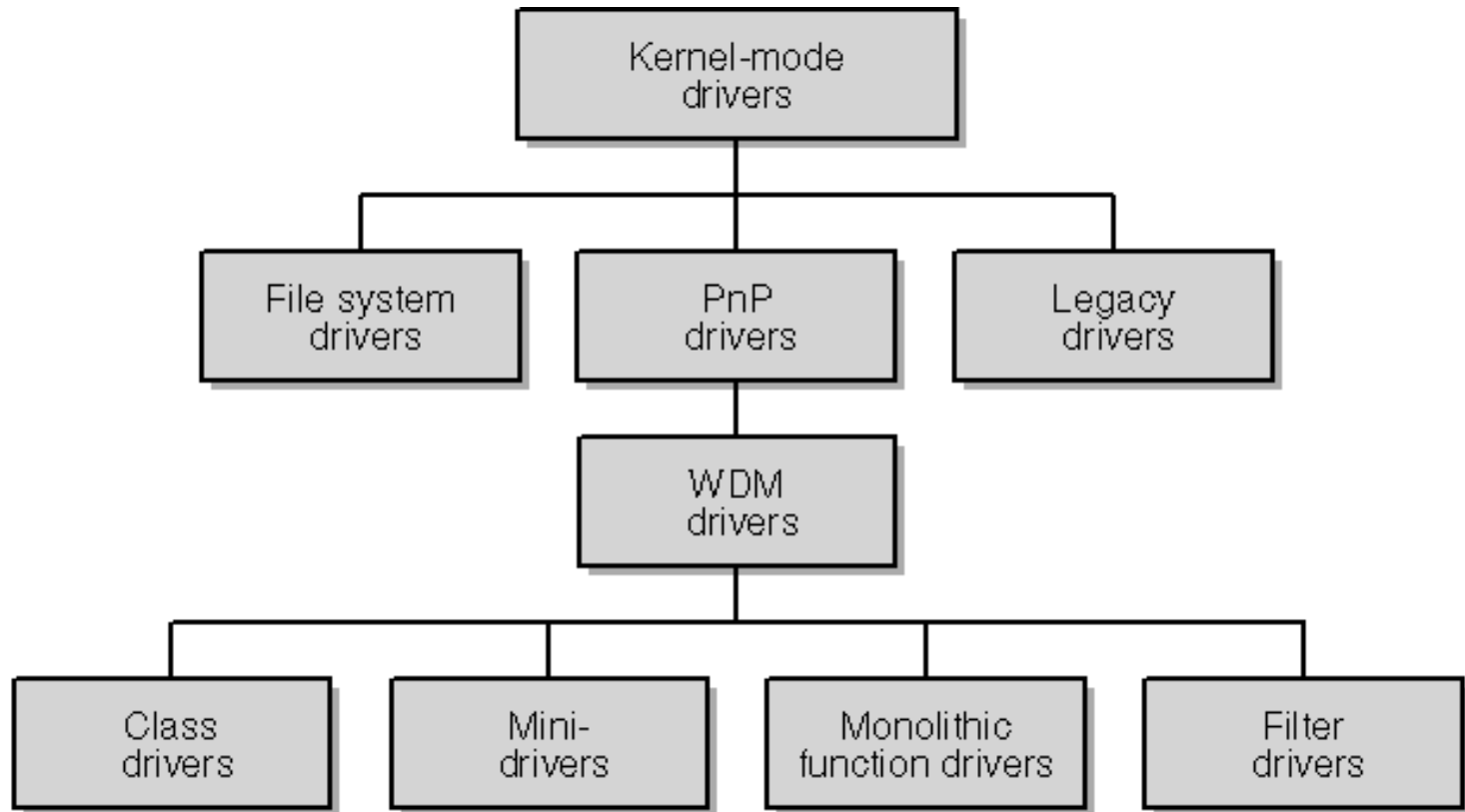
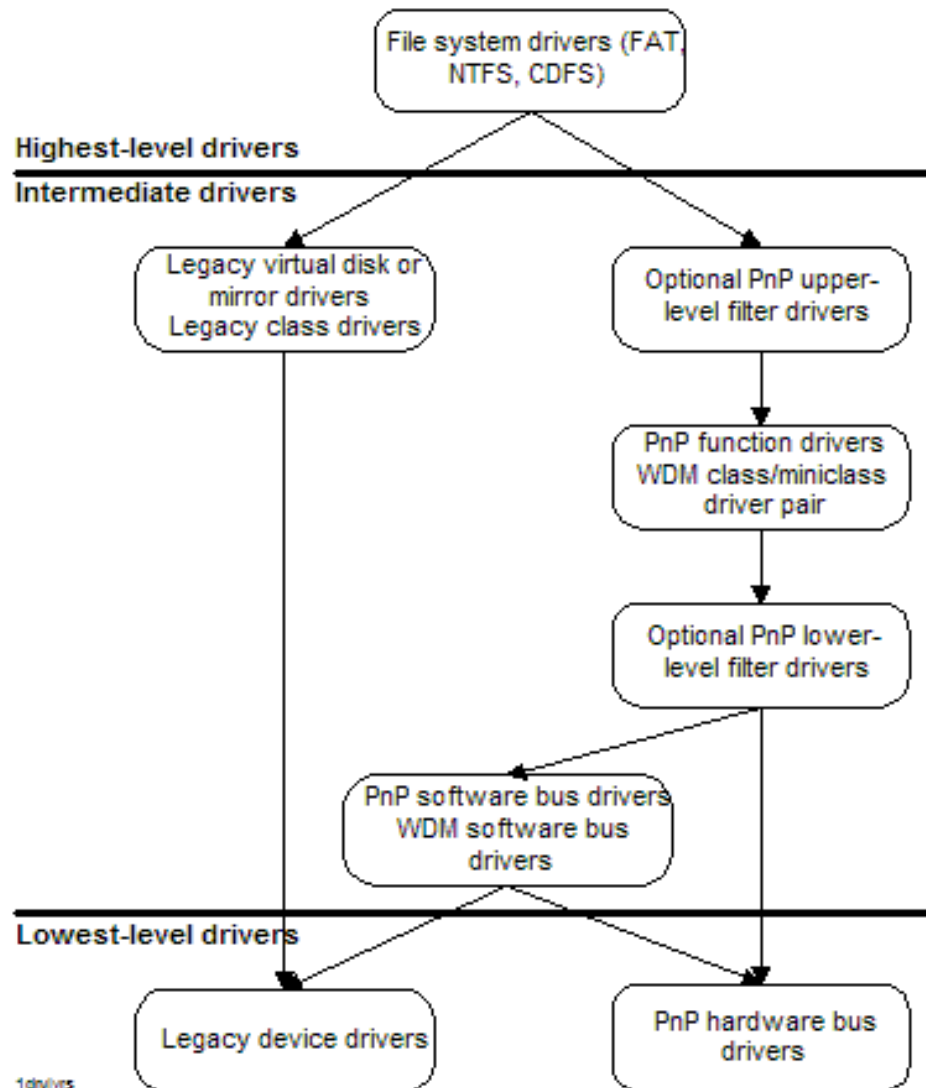


Sterowniki w systemie Windows

Klasyfikacja sterowników dla Windows XP



Klasyfikacja warstwowa sterowników



Sterowniki WDM

- Sterowniki WDM (Windows Driver Model) to sterowniki Pnp, które dodatkowo implementują protokoły zarządzania energią i są zgodne zarówno z Windows 98/ME i Windows 2K/XP.
- To definicja. W praktyce sterowniki WDM nie zawsze oferują systemowi operacyjnemu wszystkie funkcje

Sterowniki WDM

- Sterowniki klas: Łączą wspólną funkcjonalność dobrze zdefiniowanych klas urządzeń (np. kart sieciowych - NDIS class drivers). Producenci sprzętu w większości nie piszą własnych sterowników klas, ale używają jednego z dostarczonych przez Microsoft.
- Minidriver: Pisane przez producentów jako wsparcie dla sterowników klas.
- Sterowniki Filtrów (filter driver): Dodają i/lub modyfikują zachowanie urządzenia
- Sterowniki monolityczne (Monolithic function driver): Jedne sterownik dla całego urządzenia (systemy typu embedded)

Klasyfikacja sterowników dla Windows XP

- Sterowniki systemu plików: implementują system plików dla dysku lokalnego lub podłączonego sieciowo.
- Sterowniki typu Legacy: Kontrolują sprzęt bez żadnego wsparcia od strony systemu operacyjnego (Sterowniki dla nieistniejącego sprzętu są dodane do systemu właśnie jako legacy drivers)
- Sterowniki Pnp: Implementują protokół Plug and Play.

Sterowniki urządzeń dla Windows

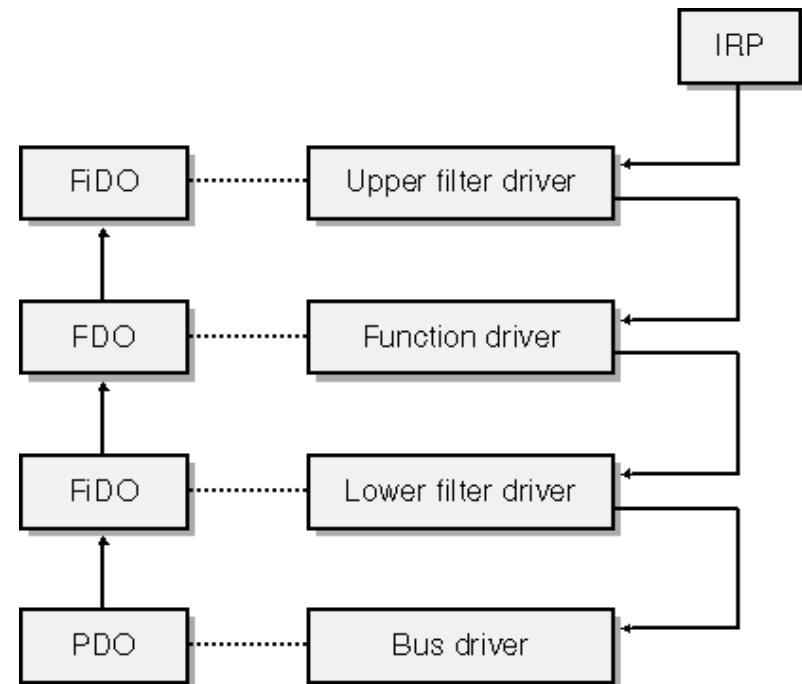
- Rzeczywisty plik sterownika, który zawiera realizację funkcjonalności urządzenia ma rozszerzenie .sys. Plik ten jest uzyskiwany po kompilacji kodu źródłowego przy użyciu pakietu DDK.
- Dla sterowników wymagany jest jeszcze plik .inf umożliwiający zainstalowanie pliku w systemie.
- Plik Inf: plik tekstowy z informacjami o urządzeniu (urządzeniach) i plikach, które należy dodać do systemu. Używany przez Win32 Installer.

Device extension i stos sterowników

- Dane specyficzne dla urządzenia (stan urządzenia, wykorzystane przez nie zasoby np. pamięć, uchwyty) są utrzymywane w strukturze Device extension.
- Warstwowe ułożenie sterowników: Przychodzące pakiety IRP są przetwarzane przez każdą pojedynczą warstwę (sterownik) i przekazywane do następnego sterownika. Jest to tzw. Stos komunikatów dla sterowników → IRP stack location

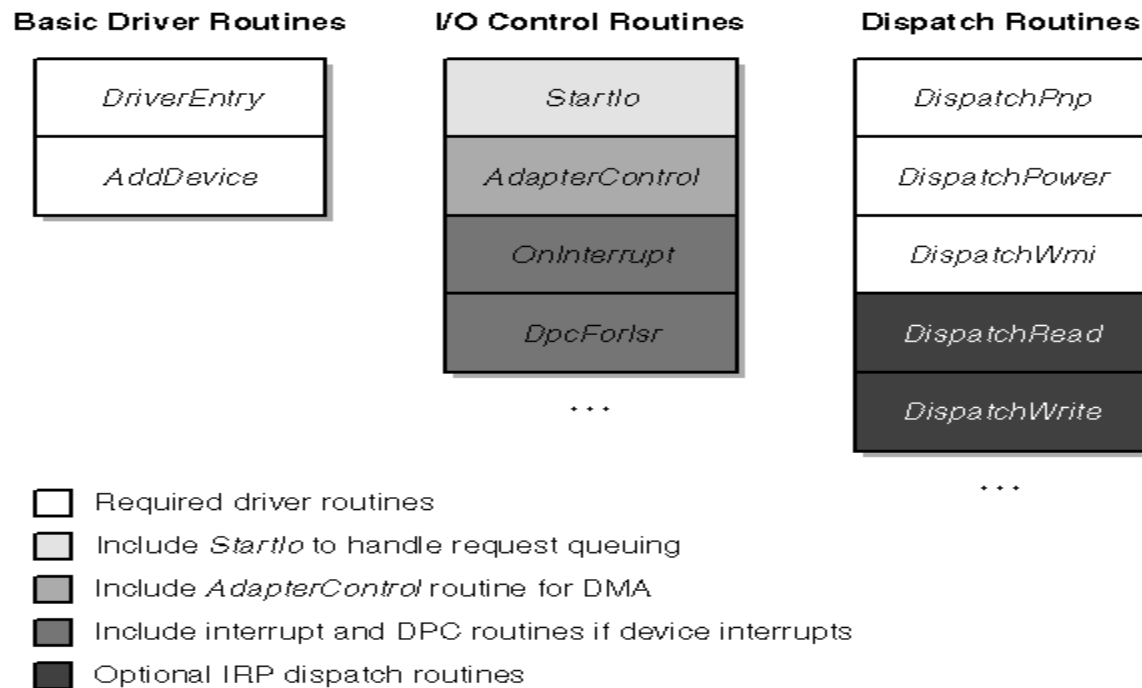
Warstwy sterowników

- Linie wykropkowane pokazują połączenia logiczne pomiędzy warstwami sterowników a strukturami device object.
- Sterowniki funkcji: Sterownik który faktycznie zarządza sprzętem.
- Sterownik magistrali (Bus driver): odpowiedzialny za zarządzanie połączeniem pomiędzy sprzętem a systemem.
- Sterowniki filtrów: modyfikują żądanie IRP (na dowolnej ścieżce)



Procedury sterownika WDM

- Architektura sterownika i obowiązkowe procedury sterownika zależą od typu urządzenia

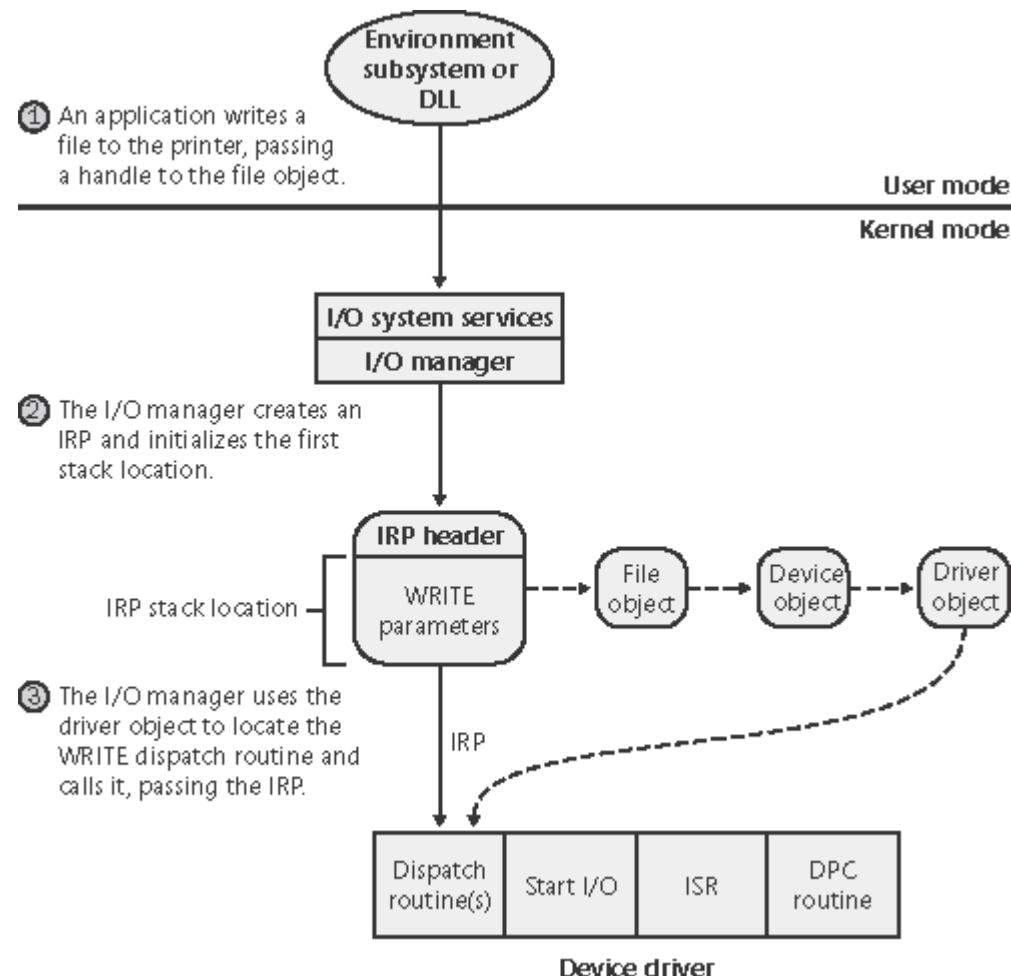


Procedury obowiązkowe

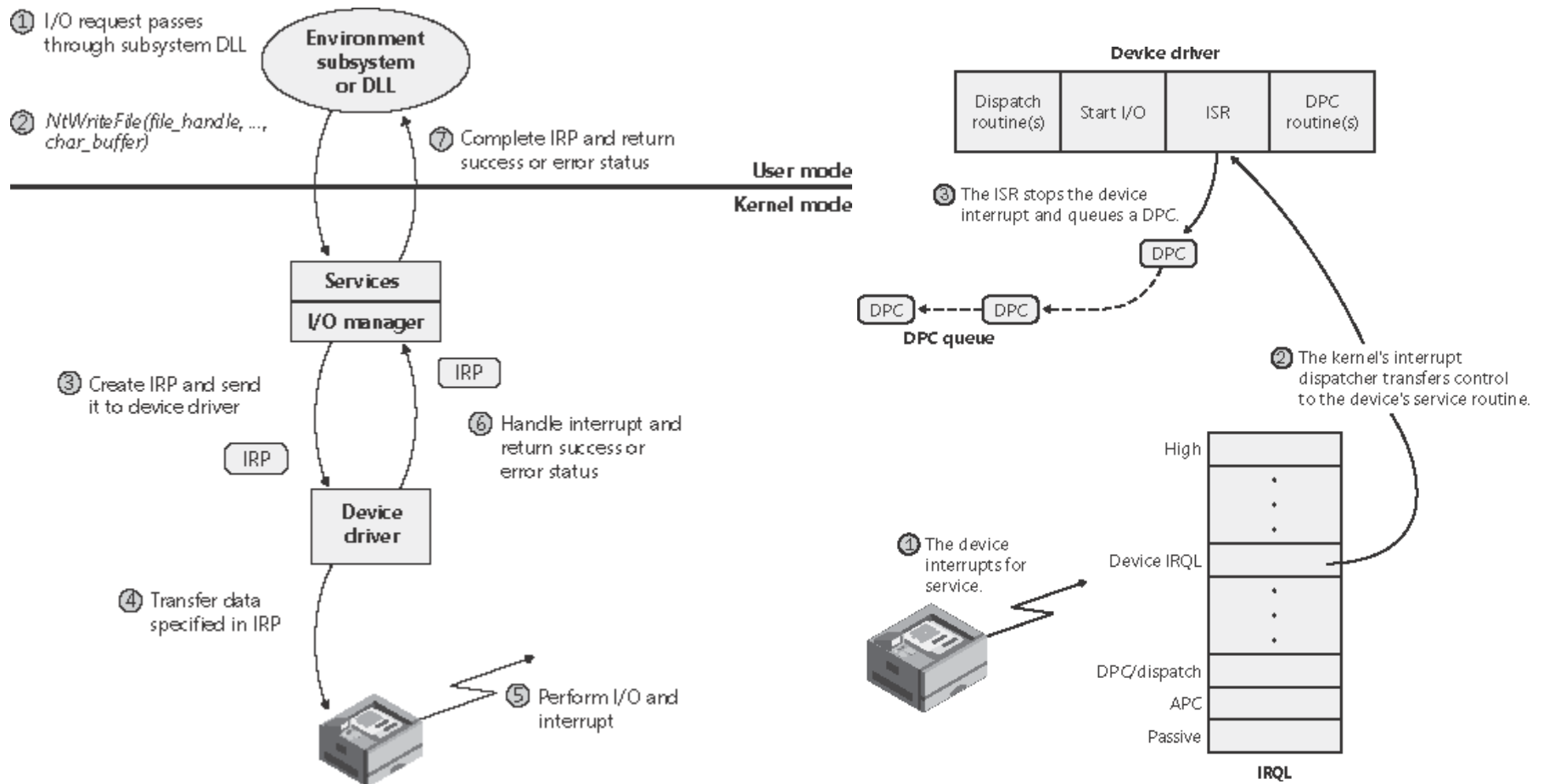
- **DriverEntry:** Funkcja inicjalizująca działanie sterownika. Jej zadanie to wskazanie SO, gdzie znajdują się niezbędne funkcje (np. AddDevice). W praktyce oznacza to wskazanie SO przez sterownik, które funkcje może użytkownik (a raczej jego aplikacja) wykonać.
- **AddDevice:**
 - Tworzy obiekt urządzenia (device object) oraz instancję rozszerzenia urządzenia (device extension)
 - Rejestruje jedno lub więcej interfejsów urządzeń
 - Wpisuje device extension i flagi (flags member) do obiektu urządzenia (device object).
 - Dołącza device object do stosu urządzeń (device stack).

Przetwarzanie pakietów IRP

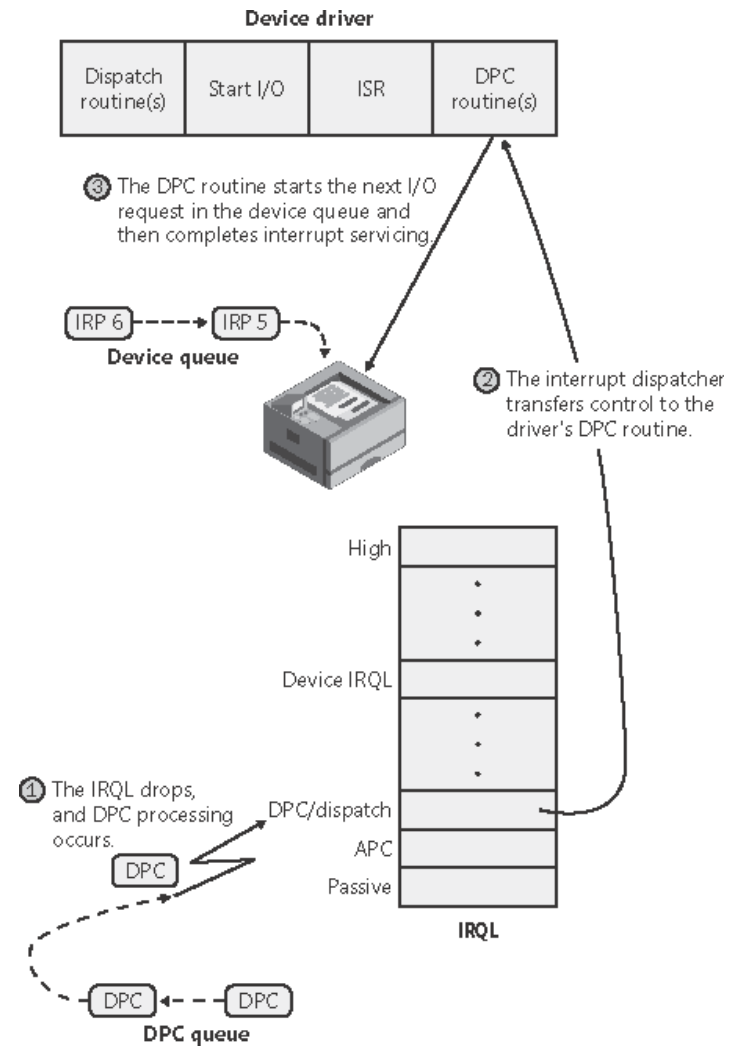
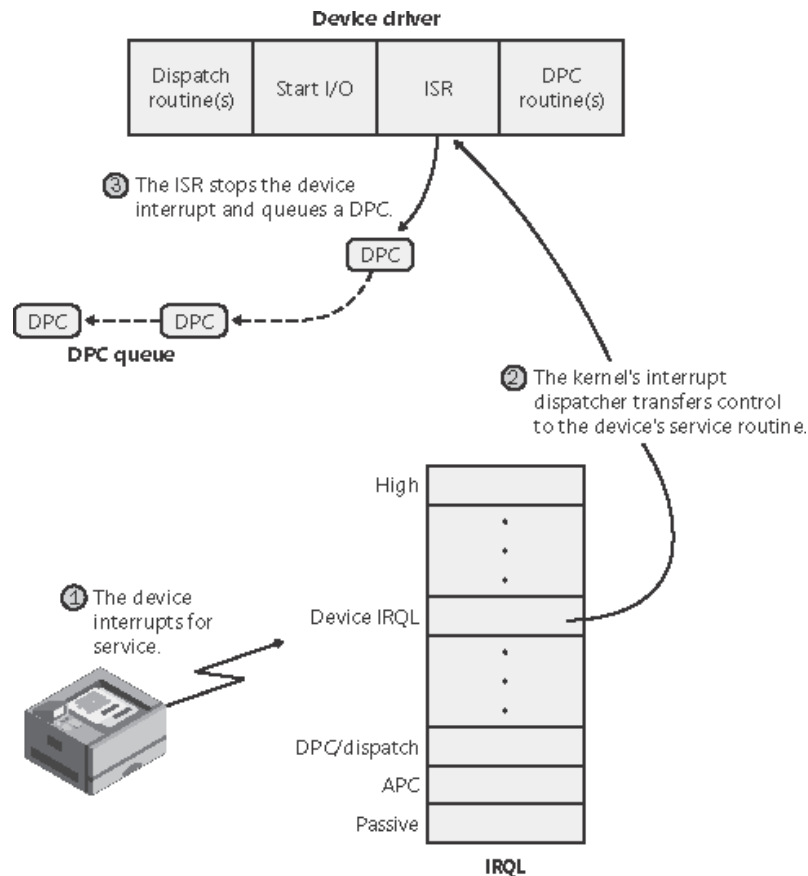
- Sterownik jednowarstwowy



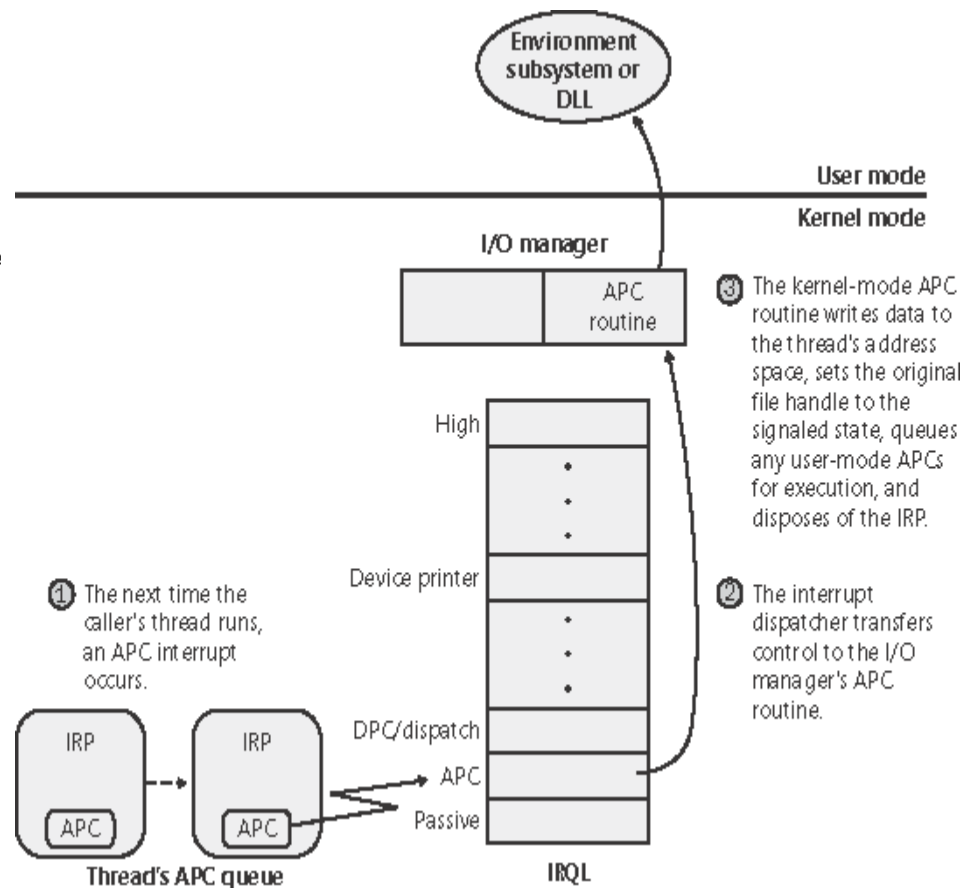
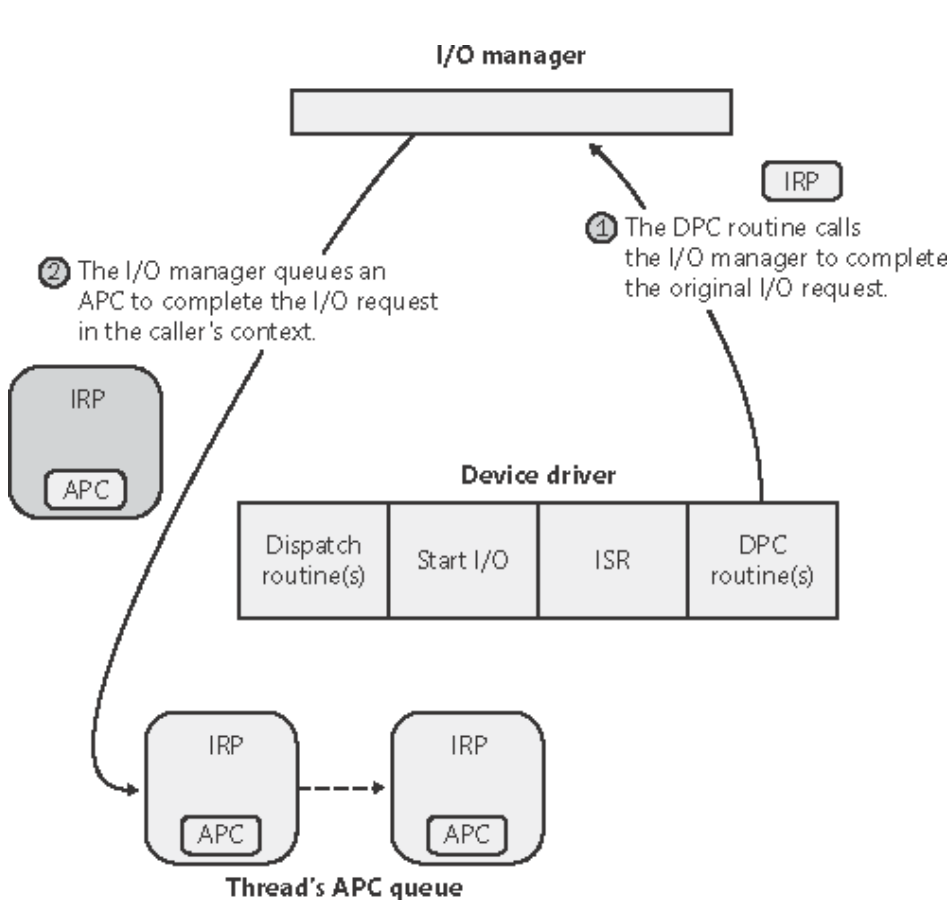
Przetwarzanie żądań synchronicznych



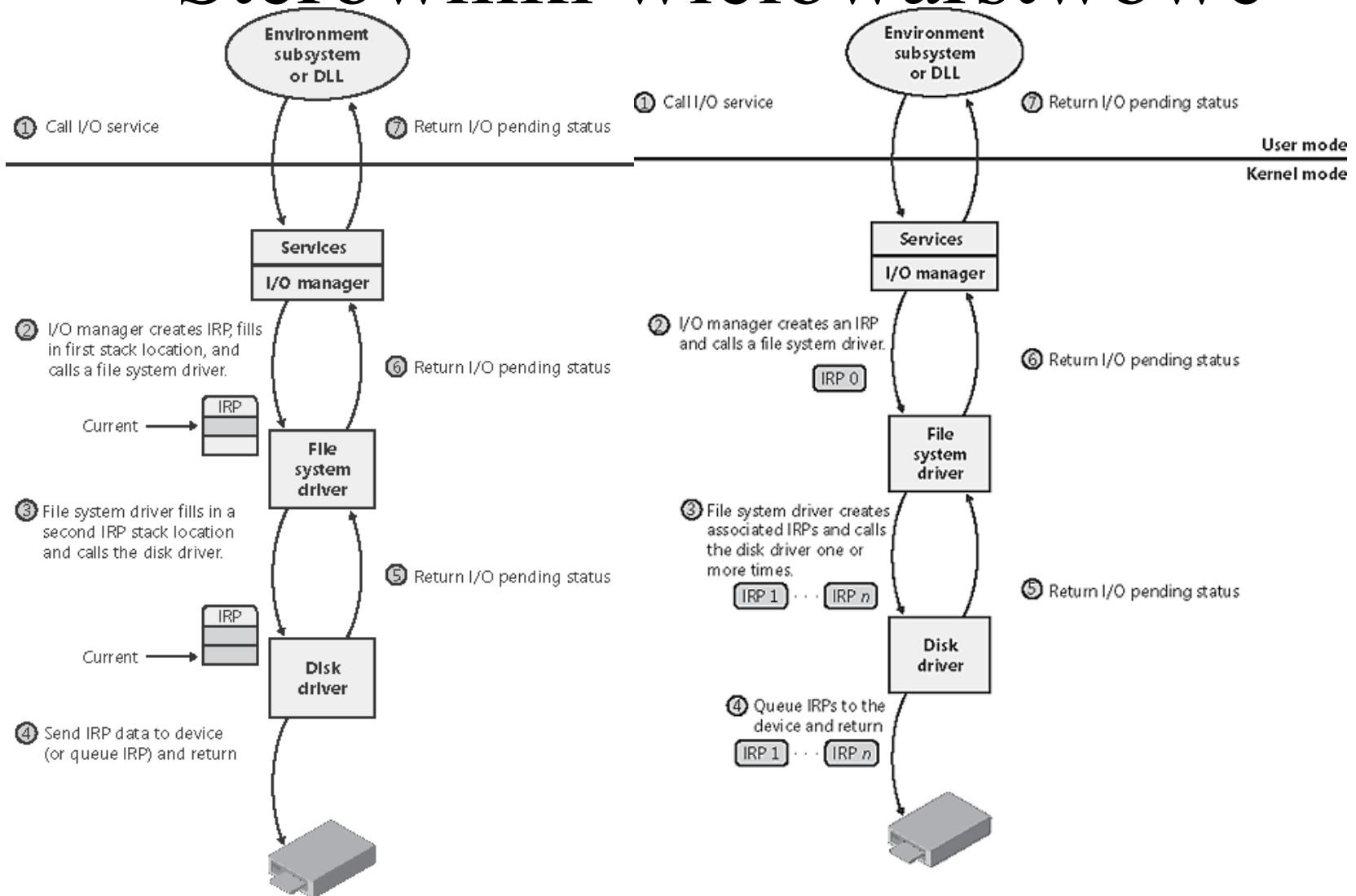
Obsługa przerwań



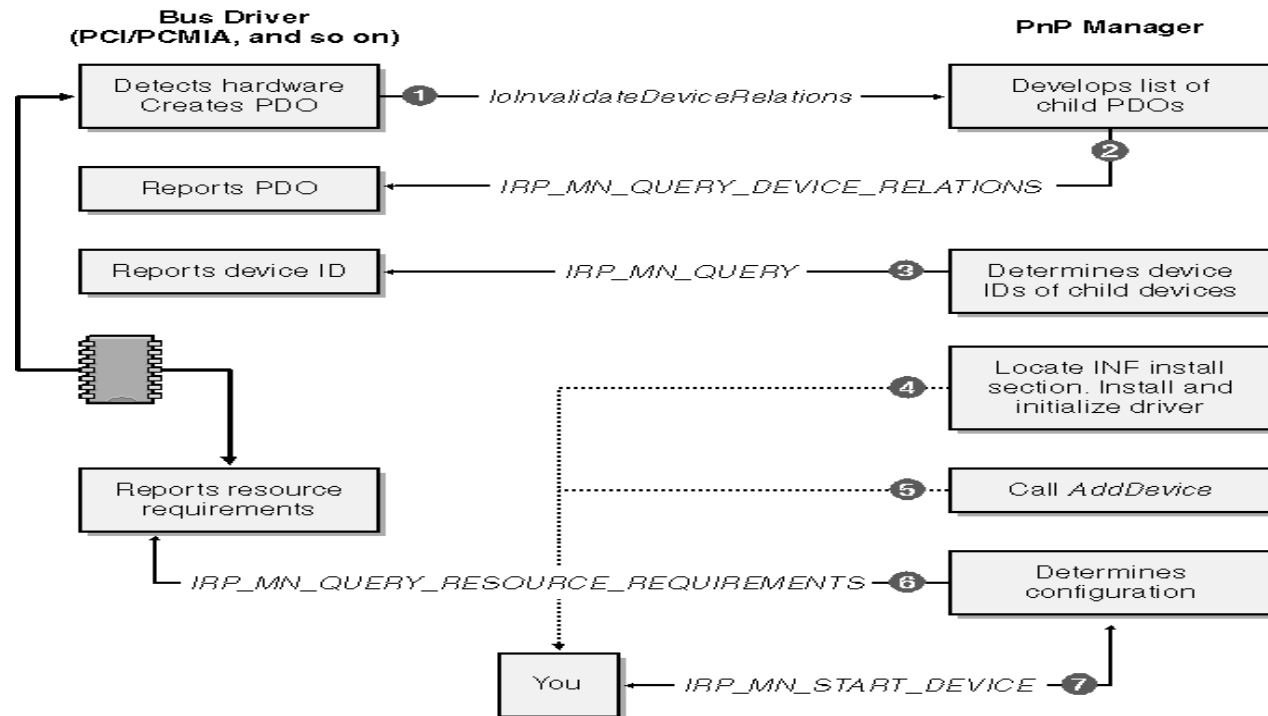
Kończenie obsługi IRP



Sterowniki wielowarstwowe



Instalacja i ładowanie sterownika PnP



Instalacja i ładowanie sterownika PnP

Sterownik magistrali (Bus driver) wykrywa hardware i pyta menadżera PnP o zaktualizowaną listę urządzeń.

Menadżer Pnp wysyła IRP (I/O request) do sterownika magistrali.

Sterownik magistrali zwraca listę znalezionych PDO (Device Object), które menadżer Pnp użył do określenia niezainicjalizowanych PDO.

Menadżer Pnp pyta sterownik magistrali o identyfikator urządzenia, który jest użyty przez menadżera PnP do lokalizacji klucza w Rejestrze systemowy

Jeśli sterownik jest już zainstalowany PnP Menadżer przechodzi do wywołania funkcji AddDevice (3 kroki dalej)

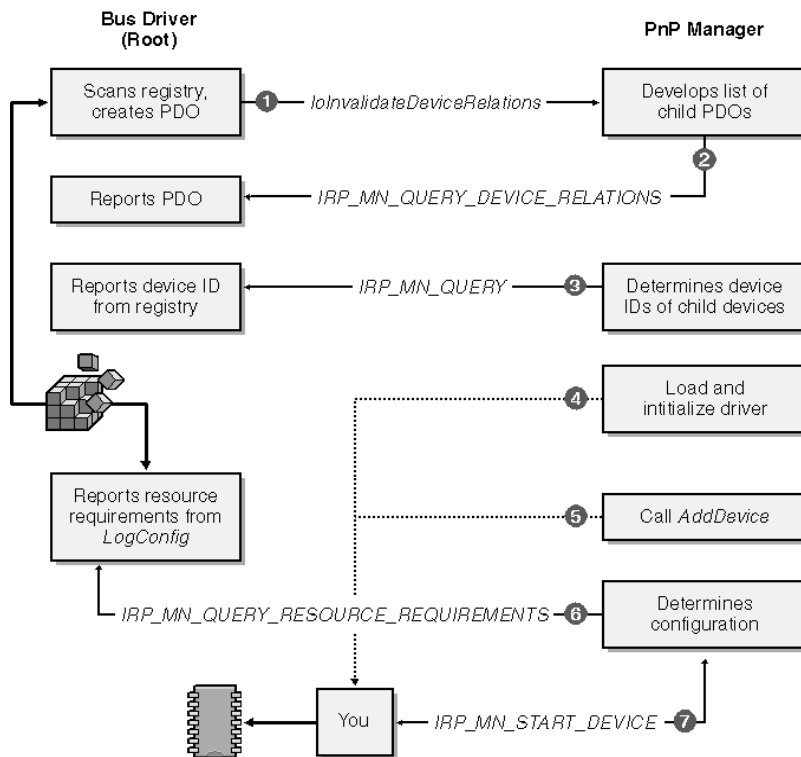
Menadżer Pnp instaluje sterownik używając właściwego pliku inf.

➔ Odtąd urządzenie jest znane przez system

Instalacja i ładowanie sterownika PnP

- Menadżer Pnp woła menadżera pamięci w celu mapowania pamięci zanim sterownik zostanie załadowany.
- Memory manager woła DriverEntry.
- Menadżer Pnp woła funkcję AddDevice sterownika, i w ten sposób załadowany sterownik jest powiadomiony o wykryciu nowego urządzenia.
- Menadżer Pnp wysyła IRP do sterownika magistrali aby pozyskać informacje o potrzebnych zasobach
- Menadżer Pnp konfiguruje sprzęt i wysyła IRP_MJ_PNP do sterownika. Urządzenie jest uruchomione.

Instalacja i ładowanie sterowników nie - PnP

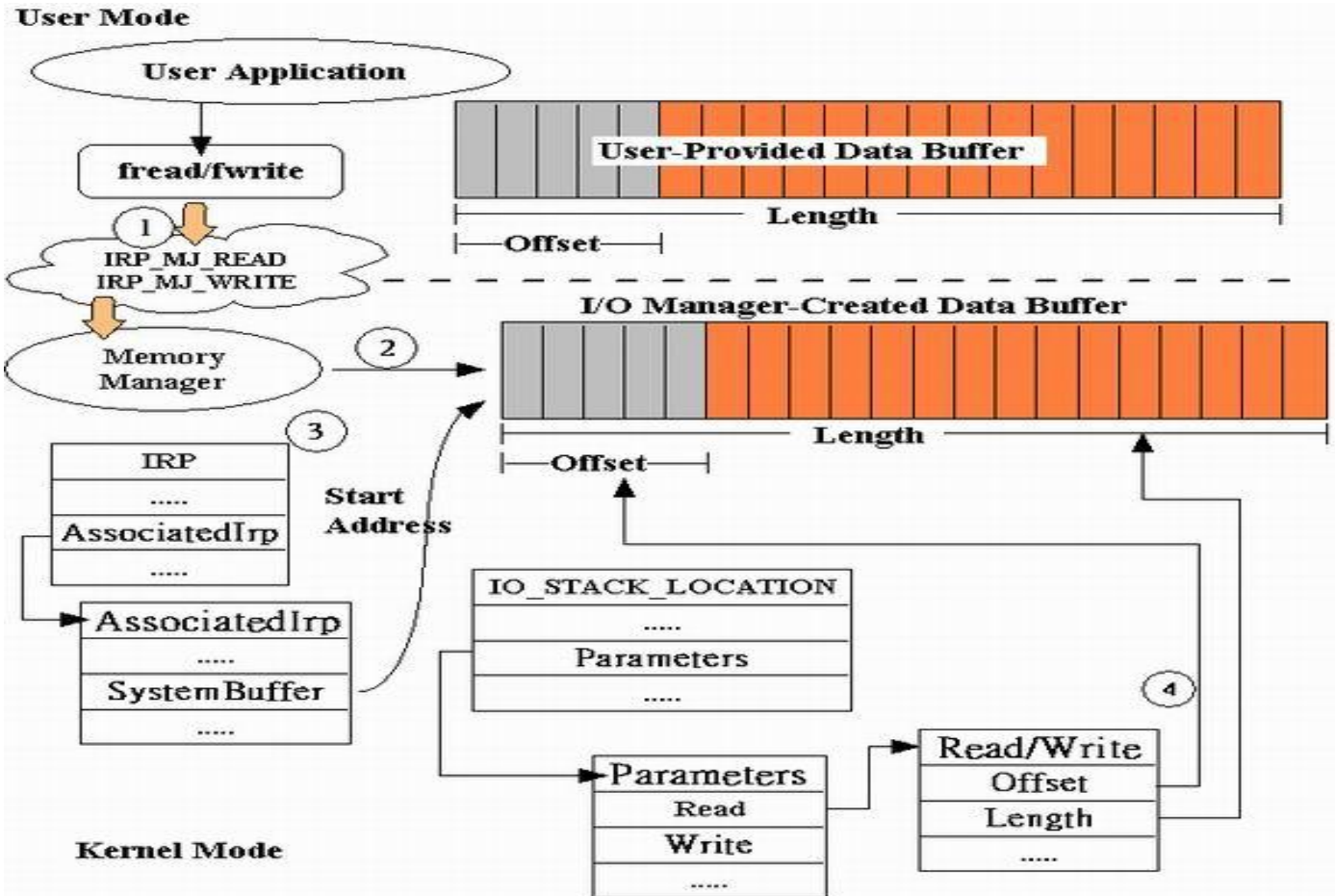


- Użytkownik instaluje sterownik (np.. Przez “Add new hardware”)
➔ Zostaje utworzony właściwy wpis w rejestrze
- System jest restartowany.
- The root enumerator skanuje rejestr i znajduje nowo dodany sprzęt.
- Po tym, proces jest identyczny jak dla sterowników Pnp.

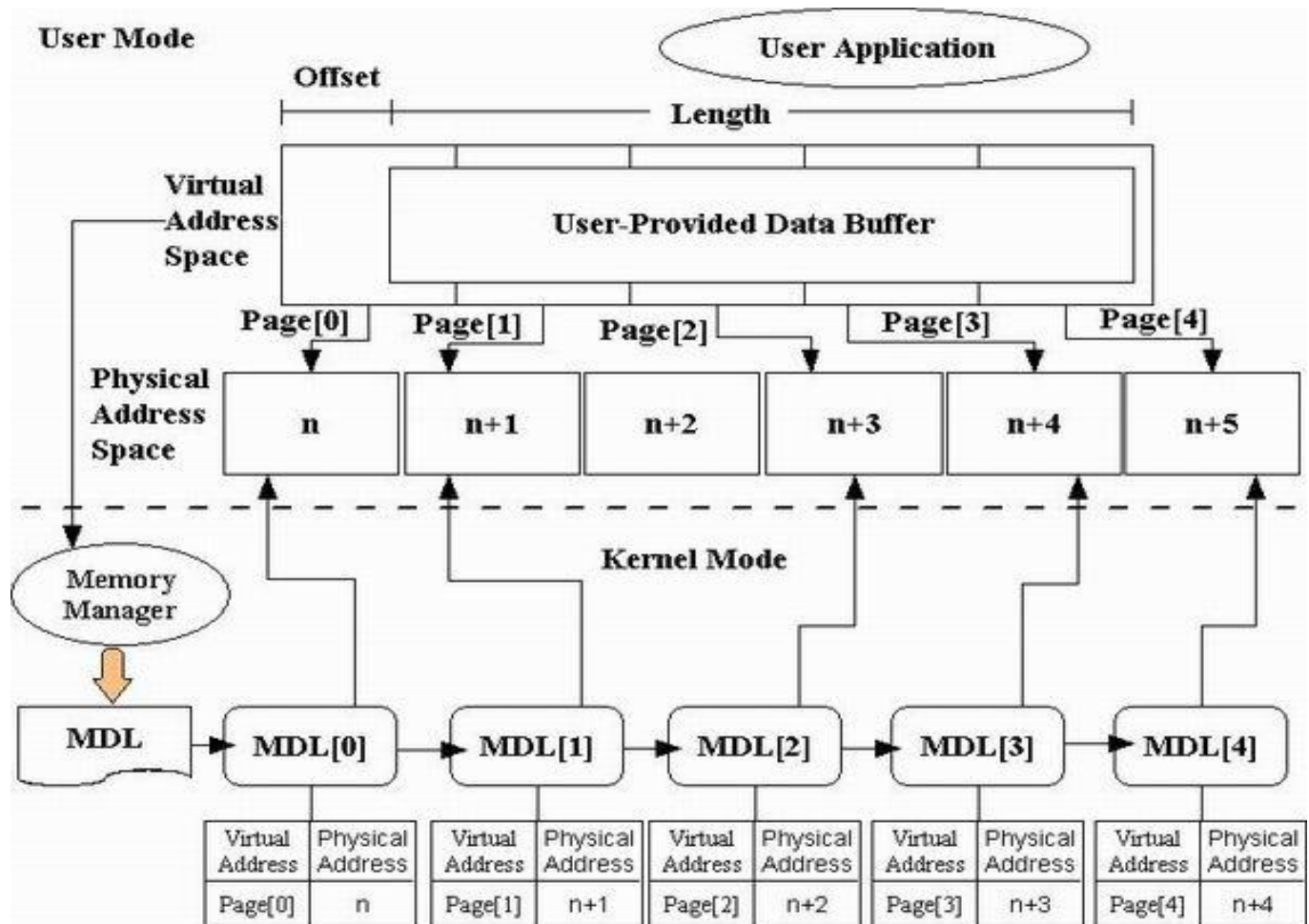
Funkcje odczytu/zapisu

- Istnieją trzy sposoby przekazywania danych pomiędzy user i kernel mode
 - Direct I/O (bezpośredni)
 - Buffered I/O (buforowany)
 - Neither (własny)

Buforowane operacje I/O



Direct I/O



Literatura

- W. Oney, *Programming the Windows Driver Model 2nd Edition*, Microsoft Press, 2003
- DDK documentation, <http://www.microsoft.com/whdc/devtools/ddk/default.mspx>
- Introduction to Drivers: Part 1, <http://www.codeproject.com/system/driverdev.asp>
- Introduction to Drivers: Part 2: <http://www.codeproject.com/system/driverdev2.asp>
- Tips for Windows NT Driver Developers -- Things to Avoid;
<http://support.microsoft.com/default.aspx?scid=KB;EN-US;Q186775&>
- <http://www.codeproject.com/system/driverdev.asp>

Sterowniki WDF

Rola WDF

- Sterowniki modelu WDF (*Windows Driver Foundation*) spełniają tę samą rolę co sterowniki modelu WDM (*Windows Driver Model*): obsługują komunikację pomiędzy systemem operacyjnym Windows i urządzeniem.
- Reprezentują całkowicie nową budowę sterowników, powiązaną z WDM, umożliwiając łatwiejszą implementację.
- Istotną rolę odgrywa dobrze zdefiniowany model obiektowy, umożliwiający sterownikom WDF komunikację z obiektami poprzez spójny interfejs.

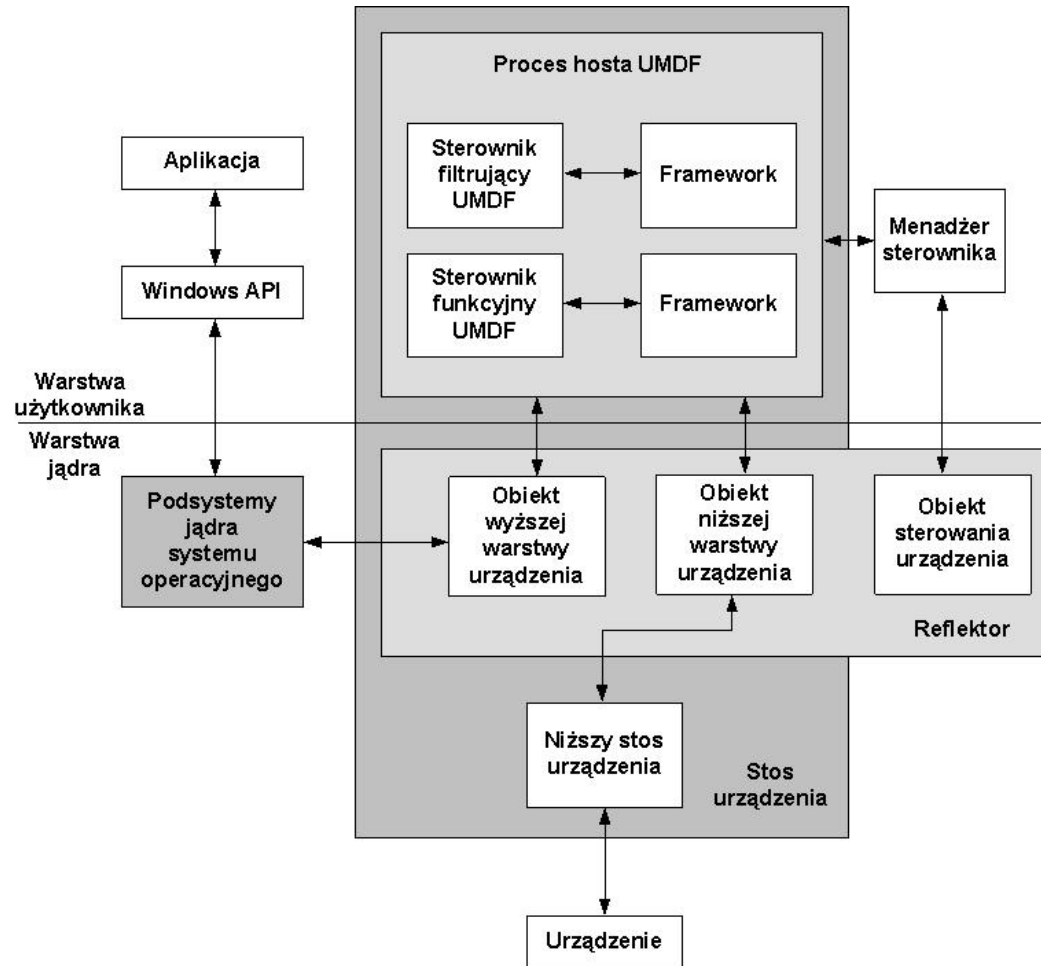
Rola WDF

- W skład modelu WDF wchodzi dwa frameworki: UMDF (User-Mode Driver Framework) i KMDF (Kernel-Mode Driver Framework), wspierające odpowiednio budowę sterowników warstwy użytkownika oraz jądra systemu operacyjnego
- Każdy z nich ma odmienną budowę.
- UMDF jest przeznaczony dla warstwy aplikacji (user mode) i ma postać biblioteki dll
- KMDF jest przeznaczony dla kernela SO i jest zgodny ze sterownikami WDM

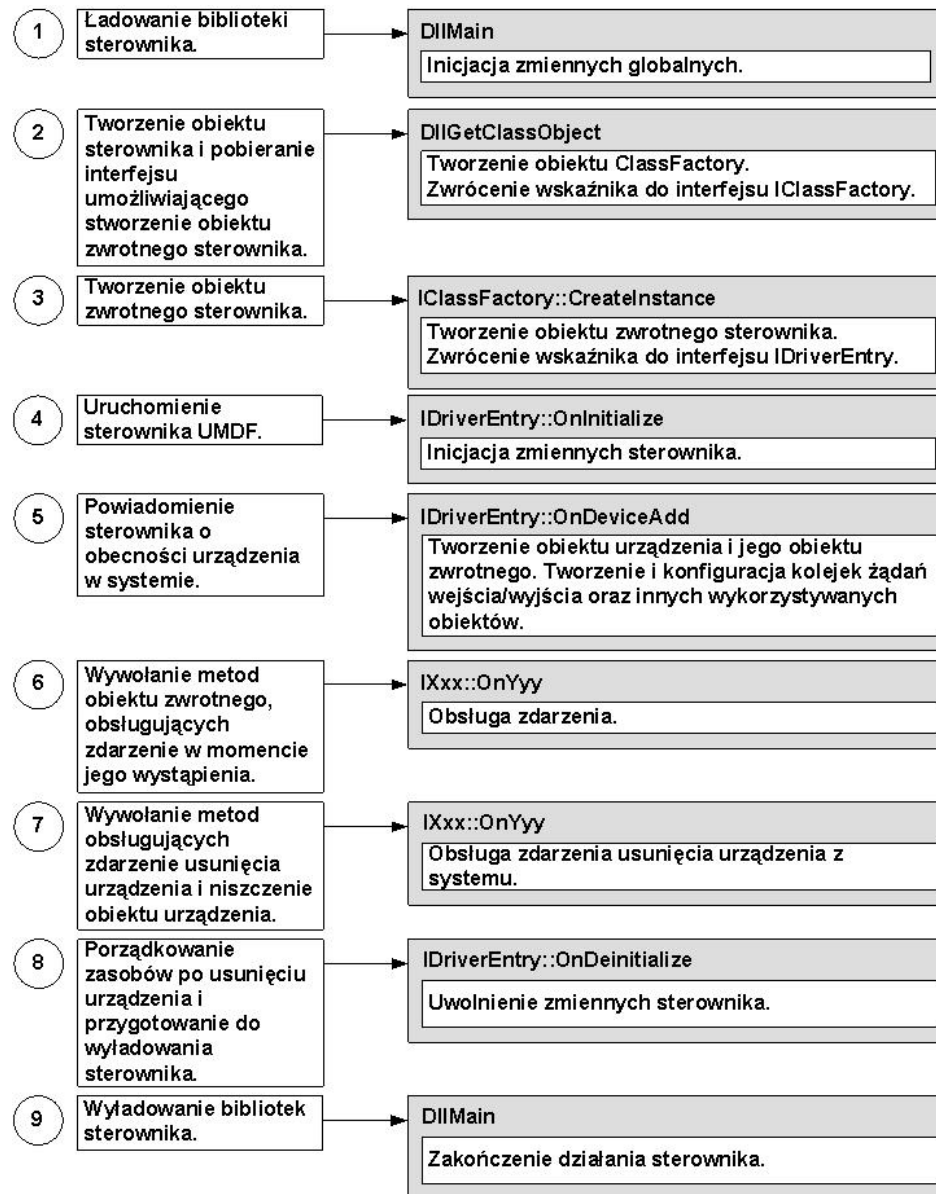
UMDF

- UMDF jest modelem programowania opartym na modelu COM.
- Umożliwiającym budowę sterowników funkcyjnych i filtrujących dla urządzeń bazujących na protokole „Plug and Play”, np. urządzeń USB.
- Dostarcza on wiele podstawowych obiektów, należących do dwóch głównych kategorii:
 - obiekty frameworka (ang. framework objects) - są to obiekty wbudowane w framework, reprezentujące: sterownik, obsługiwane przez niego urządzenie, kolejkę wejścia/wyjścia żądań, itp, które mogą być tworzone zarówno przez framework jak i sterownik. Każdy z obiektów posiada odpowiednie metody, pozwalające mu na wykonanie danej akcji, pobranie lub zapisanie wartości jego właściwości.
 - obiekty zwrotne (ang. callback objects) - są to obiekty implementowane przez sterownik i rejestrowane we frameworku. Umożliwiają one stworzenie własnej, specyficznej dla danego urządzenia obsługi zdarzenia, wywołanego przez obiekty frameworka. Udostępniają jeden lub więcej interfejsów, z których każdy posiada zbiór metod, obsługujących poszczególne zdarzenia. W momencie zaistnienia w systemie operacyjnym określonych zmian, np. zmiany stanu zasilania, framework wywołuje je w celu powiadomienia sterownika o ich wystąpieniu i przekazaniu do niego odpowiednich danych. Sterownik rejestruje obiekt zwrotny poprzez przekazanie do frameworka wskaźnika do niego.

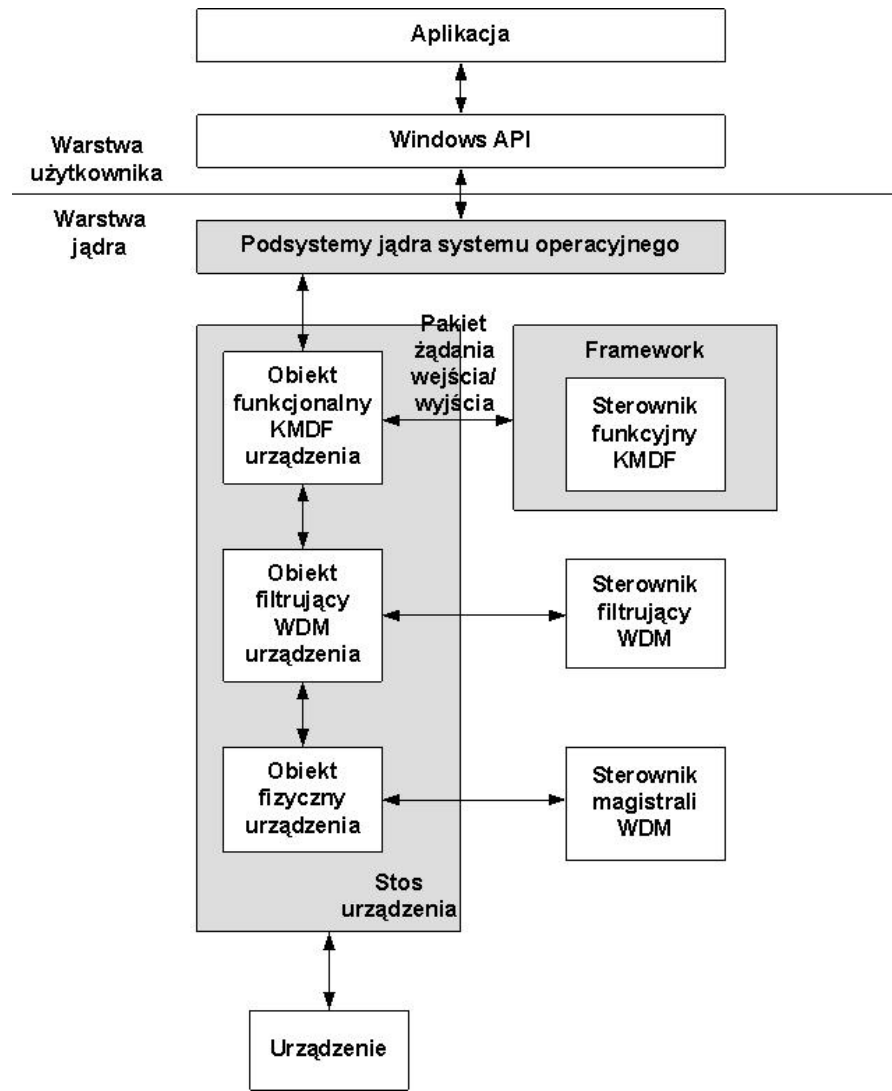
Architektura UMDF



Struktura sterownika UMDF



Architektura KMDF



Struktura sterownika KMDF

